Design and Analysis of Algorithms

Assignment Solution (FALL 2026)

Q1. Insertion Sort [2+2+2+2 Marks]

Initial array: [18, 7, 25, 14, 3, 21, 10, 30]

(a) Steps of Insertion Sort

Pass (i)	Key	Elements shifted.	Array after pass
1	7	18	[7, 18, 25, 14, 3, 21, 10, 30]
2	25	none	[7, 18, 25, 14, 3, 21, 10, 30]
3	14	25, 18	[7, 14, 18, 25, 3, 21, 10, 30]
4	3	25, 18, 14, 7	[3, 7, 14, 18, 25, 21, 10, 30]
5	21	25	[3, 7, 14, 18, 21, 25, 10, 30]
6	10	25, 21, 18, 14	[3, 7, 10, 14, 18, 21, 25, 30]
7	30	none	[3, 7, 10, 14, 18, 21, 25, 30]

Final sorted array: [3, 7, 10, 14, 18, 21, 25, 30]

(b) Why the worst case is $O(n^2)$

The worst case occurs when the array is in reverse (descending) order. Then every key is smaller than all elements before it, so in pass i, the inner loop must compare and shift all previous elements.

$$\text{Total comparisons/shifts} = 1 + 2 + 3 + \ldots + (n-1) = n(n-1)/2$$

Since $n(n-1)/2 = (n^2 - n)/2$, the dominant term is n^2, so the worst-case running time is **$O(n^2)$**.

(c) Correctness using a loop invariant

Loop invariant: At the start of each iteration of the outer loop (for index i), the subarray A[0 … i−1] contains the elements originally in A[0 … i−1], but in sorted order.

1. **Initialization:** Before the first iteration (i = 1), the subarray A[0 … 0] has only one element, which is trivially sorted. So the invariant holds.
2. **Maintenance:** During each iteration, the key A[i] is compared with elements of the sorted subarray from right to left. Larger elements are shifted one position right, and the key is inserted into its correct place. Hence A[0 … i] becomes sorted, and the invariant holds for the next iteration.
3. **Termination:** The loop ends when i = n. By the invariant, A[0 … n−1], which is the entire array, is sorted. Therefore, Insertion Sort correctly sorts the array.

Solution

Q2. Merge Sort [5+3 Marks]

Initial array: [32, 15, 8, 24, 11, 40, 6, 19]

(a1) Division steps

The array is repeatedly split into halves until each subarray has one element.

```
Level 0:              [32, 15, 8, 24, 11, 40, 6, 19]
                           /                    \
Level 1:        [32, 15, 8, 24]            [11, 40, 6, 19]
                  /        \                  /        \
Level 2:     [32, 15]    [8, 24]         [11, 40]    [6, 19]
               /   \       /   \           /   \       /   \
Level 3:   [32]  [15]   [8]  [24]      [11]  [40]   [6]  [19]
```

(a2) Recursive sorting and merging steps

Single elements are already sorted, so merging begins from the bottom level.

Merge level 3 → 2 (pairs):

Subarrays merged	Result
[32] + [15]	[15, 32]
[8] + [24]	[8, 24]
[11] + [40]	[11, 40]
[6] + [19]	[6, 19]

Merge level 2 → 1:

Subarrays merged	Comparisons	Result
[15, 32] + [8, 24]	8<15 → 8; 15<24 → 15; 24<32 → 24; then 32	[8, 15, 24, 32]
[11, 40] + [6, 19]	6<11 → 6; 11<19 → 11; 19<40 → 19; then 40	[6, 11, 19, 40]

Merge level 1 → 0 (final merge): [8, 15, 24, 32] + [6, 11, 19, 40]

6 < 8 → 6; 8 < 11 → 8; 11 < 15 → 11; 15 < 19 → 15; 19 < 24 → 19; 24 < 40 → 24; 32 < 40 → 32; then 40.

Final sorted array: [6, 8, 11, 15, 19, 24, 32, 40]

(b) Running time of 3-way Merge Sort

$$T(n) = 3T(n/3) + \Theta(n)$$

1. **Divide:** Splitting the array into three parts takes constant time, $\Theta(1)$.
2. **Conquer:** Three subproblems, each of size n/3, give $3T(n/3)$.
3. **Combine:** The three-way merge picks the smallest of the three front elements at each step, which takes at most 2 comparisons (a constant). Since n elements are output, merging takes $\Theta(n)$.

Solution

Master Theorem: a = 3, b = 3, f(n) = Θ(n), and n^(log₃ 3) = n. Since f(n) = Θ(n^(log_b a)), Case 2 applies, so T(n) = Θ(n log n).

Intuition: The recursion tree has log₃ n levels, and each level does Θ(n) merging work, giving Θ(n log₃ n) = **Θ(n log n)**, because log₃ n and log₂ n differ only by a constant factor. The asymptotic time is the same as 2-way Merge Sort; only the constant factor changes.

Q3. Quick Sort (first element as pivot) [2.5 + 2.5 + 3 Marks]

Initial array: [24, 9, 17, 5, 31, 12, 27, 3]

Partition procedure used:

```
PARTITION(A, low, high):
    pivot = A[low]
    i = low
    for j = low + 1 to high:
        if A[j] < pivot:
            i = i + 1
            swap A[i] and A[j]
    swap A[low] and A[i]
    return i
```

(a) Detailed trace of the first partition (pivot = 24)

j	A[j]	A[j] < 24?	Action	Array
1	9	Yes	i = 1, swap A[1], A[1]	[24, 9, 17, 5, 31, 12, 27, 3]
2	17	Yes	i = 2, swap A[2], A[2]	[24, 9, 17, 5, 31, 12, 27, 3]
3	5	Yes	i = 3, swap A[3], A[3]	[24, 9, 17, 5, 31, 12, 27, 3]
4	31	No	none	[24, 9, 17, 5, 31, 12, 27, 3]
5	12	Yes	i = 4, swap 31 and 12	[24, 9, 17, 5, 12, 31, 27, 3]
6	27	No	none	[24, 9, 17, 5, 12, 31, 27, 3]
7	3	Yes	i = 5, swap 31 and 3	[24, 9, 17, 5, 12, 3, 27, 31]

Final swap A[0] ↔ A[5] (24 ↔ 3): **[3, 9, 17, 5, 12, 24, 27, 31]**. Pivot 24 is now at index 5.

(b) Array after each major partition step

Step	Subarray partitioned	Pivot	Array after partition	Pivot position	Remaining subarrays
1	[24, 9, 17, 5, 31, 12, 27, 3]	24	[3, 9, 17, 5, 12, 24, 27, 31]	index 5	[3, 9, 17, 5, 12] and [27, 31]
2	[3, 9, 17, 5, 12]	3	[3, 9, 17, 5, 12, 24, 27, 31]	index 0	[ ] and [9, 17, 5, 12]
3	[9, 17, 5, 12]	9	[3, 5, 9, 17, 12, 24, 27, 31]	index 2	[5] and [17, 12]
4	[17, 12]	17	[3, 5, 9, 12, 17, 24, 27, 31]	index 4	[12] and [ ]
5	[27, 31]	27	[3, 5, 9, 12, 17, 24, 27, 31]	index 6	[ ] and [31]

Subarrays of size 0 or 1 are already sorted, so no further partitioning is needed.

Final sorted array: [3, 5, 9, 12, 17, 24, 27, 31]

(c) Loop invariant for the Partition procedure

Loop invariant: At the start of each iteration of the for loop (with index j):

1. A[low] = pivot.
2. Every element in A[low+1 … i] is less than the pivot.
3. Every element in A[i+1 … j−1] is greater than or equal to the pivot.
4. Elements in A[j … high] have not been examined yet.

Initialization: Before the first iteration, i = low and j = low + 1. Both regions A[low+1 … i] and A[i+1 … j−1] are empty, so conditions 2 and 3 hold trivially. Condition 1 holds because the pivot was taken from A[low].

Maintenance: If A[j] ≥ pivot, only j is incremented and A[j] joins the "≥ pivot" region. If A[j] < pivot, i is incremented and A[i] is swapped with A[j]: the smaller element moves into the "< pivot" region, and the element it replaces (which was ≥ pivot) moves to the end of the "≥ pivot" region. In both cases the invariant still holds.

Termination: The loop ends when j = high + 1, so every element has been examined: A[low+1 … i] < pivot and A[i+1 … high] ≥ pivot. The final swap exchanges the pivot with A[i], which is either less than the pivot or the pivot itself. Now all elements to the left of index i are less than the pivot and all elements to its right are greater than or equal to it, so the pivot is in **its correct final position**.

Q4. Dividing 2n Players into Two Balanced Teams [4+4 Marks]

Ratings: 10, 15, 7, 20, 12, 18, 9, 14 (n = 4, total = 105, ideal ≈ 52.5 per team)

(a) $O(n^2)$ Algorithm: Greedy Maximum Selection

Idea: Repeatedly pick the highest remaining rating and give it to the team with the smaller total (if that team is not full).

```
repeat 2n times:
    find max remaining rating x
    assign x to the team with the smaller sum (and fewer than n players).
```

Example: Picks in order 20, 18, 15, 14, 12, 10, 9, 7

Pick	Rating	Assigned to	Sum A	Sum B
1	20	A	20	0
2	18	B	20	18
3	15	B	20	33
4	14	A	34	33
5	12	B	34	45
6	10	A	44	45
7	9	A (now full)	53	45
8	7	B	53	52

- Team A = {20, 14, 10, 9} = **53**
- Team B = {18, 15, 12, 7} = **52**

Running time: 2n iterations × O(n) scan = **$O(n^2)$**

Solution

(b) O(n log n) Algorithm: Sort + Snake Assignment

Strategy: Sort ratings in descending order, then assign them in the pattern **A, B, B, A, A, B, B, A, …** Each team alternately gets the stronger player of a pair, so the advantages cancel out, and each team gets exactly n players.

Example: Sorted: 20, 18, 15, 14, 12, 10, 9, 7

Rating	20	18	15	14	12	10	9	7
Team	A	B	B	A	A	B	B	A

- Team A = {20, 14, 12, 7} = **53**
- Team B = {18, 15, 10, 9} = **52**

Running time:

- Sorting: O(n log n)
- Assignment pass: O(n)
- **Total = O(n log n)**

Q5. Pairing to Minimize the Largest Pair Sum [3+5 Marks]

(a) Optimal pairing for 2, 4, 6, 8, 10, 12

Pair the smallest remaining number with the largest remaining number:

Pair	Sum
(2, 12)	14
(4, 10)	14
(6, 8)	14

Maximum pair sum = 14

This is optimal because the total is 42, so the average pair sum is 42 / 3 = 14. The largest pair sum can never be less than the average.

(b) O(n log n) Algorithm

```
MIN_MAX_PAIRING(A[1 … 2n]):
    sort A in ascending order              // O(n log n)
    maxSum = −∞
    for i = 1 to n:                        // O(n)
        pair A[i] with A[2n − i + 1]
        maxSum = max(maxSum, A[i] + A[2n − i + 1])
    return pairs, maxSum
```

Running time: Sorting takes O(n log n) and the pairing loop takes O(n), so the total is **O(n log n)**.

Why the strategy is optimal (Exchange Argument)

Let the sorted numbers be $a_1 \le a_2 \le \ldots \le a_{2n}$.

Claim: Some optimal pairing pairs a_1 (smallest) with a_{2n} (largest).

Proof: Take any optimal pairing where a_1 is paired with x and a_{2n} is paired with y. The old sums are a_1 + x and y + a_{2n}. Swap partners to form (a_1, a_{2n}) and (x, y):

Solution

$a_1 + a_{2n} \leq y + a_{2n}$, because $a_1 <= y$
$x + y \leq y + a_{2n}$ because $x <= a_{2n}$

Both new sums are at most $y + a_{2n}$, which was already one of the old sums. So the maximum pair sum does not increase, and the new pairing is still optimal.

Removing a_1 and a_{2n} leaves the same problem on the remaining $2n - 2$ numbers. Applying the same argument repeatedly (induction) shows that pairing smallest with largest at every step gives an **optimal pairing**.

Q6. Big-O Proofs [4+4 Marks]

Definition: f(n) = O(g(n)) if there exist positive constants c and n_0 such that $0 \leq f(n) \leq c \cdot g(n)$ for all $n \geq n_0$.

(a) Prove $n^2 + 5n + 10 = O(n^2)$

For $n \geq 1$, we have $n \leq n^2$ and $1 \leq n^2$.

Therefore, $5n \leq 5n^2$ and $10 \leq 10n^2$. Adding these to n^2:

$$n^2 + 5n + 10 \leq n^2 + 5n^2 + 10n^2 = 16n^2$$

Also, $n^2 + 5n + 10 \geq 0$. So $0 \leq n^2 + 5n + 10 \leq 16n^2$ for all $n \geq 1$.

$c = 16$, $n_0 = 1$. Hence $n^2 + 5n + 10 = O(n^2)$.

(Alternative: $c = 2$, $n_0 = 7$, since $5n + 10 \leq n^2$ holds for all $n \geq 7$.)

(b) Prove $4n^2 + 3n = O(n^2)$

For $n \geq 1$, we have $n \leq n^2$, so $3n \leq 3n^2$.

Therefore:

$$4n^2 + 3n \leq 4n^2 + 3n^2 = 7n^2$$

Also, $4n^2 + 3n \geq 0$. So $0 \leq 4n^2 + 3n \leq 7n^2$ for all $n \geq 1$.

$c = 7$, $n_0 = 1$. Hence $4n^2 + 3n = O(n^2)$.

(Alternative: $c = 5$, $n_0 = 3$, since $3n \leq n^2$ holds for all $n \geq 3$.)

Q7. Asymptotic Notations [2+2+1+1+2 Marks]

The open-ended question can be answered based on each student's own perspective and understanding of the video.

Q8. Master Theorem [4+4+4 Marks]

1. $$T(n) = \sqrt{2}T(n/2) + \log n \implies T(n) = \Theta(\sqrt{n}) \text{ (Case 1)}$$

2. $$16T(n/4) + n! \implies T(n) = \Theta(n!) \text{ (Case 3)}$$

Solution

3. $T(n) = 64T(n/8) - n^2 \log n \implies$ Does not apply ($f(n)$ is not positive)

Q9. Iteration Method [4+4 Marks]

Solution

$T(n) = 2T(n/2) + n\log n$

$$T(n) = \begin{cases} 1, & n == 1 \\ 2T(n/2) + n\log n & n > 1 \end{cases}$$

$$T\left(\frac{n}{2}\right) = 2\left[2T\left(\frac{n}{2^2}\right) + \frac{n}{2}\log\frac{n}{2}\right] + n\log n$$

$$T\left(\frac{n}{2}\right) = 2T\left(\frac{n}{2^2}\right) + \frac{n}{2}\log\frac{n}{2}$$

$$T\left(\frac{n}{2}\right) = 2^2 T\left(\frac{n}{2^2}\right) + n\log\frac{n}{2} + n\log n$$

$$T\left(\frac{n}{2^2}\right) = 2T\left(\frac{n}{2^3}\right) + \frac{n}{2^2}\log\frac{n}{2^2}$$

$$T\left(\frac{n}{2^2}\right) = 2^2\left[2T\left(\frac{n}{2^3}\right) + \frac{n}{2^2}\log\frac{n}{2^2}\right] + n\log\frac{n}{2} + n\log n$$

$$T\left(\frac{n}{2^2}\right) = 2^3 T\left(\frac{n}{2^3}\right) + n\log\frac{n}{2^2} + n\log\frac{n}{2} + n\log n$$

$$T\left(\frac{n}{2^3}\right) = 2^3\left[2T\left(\frac{n}{2^4} + \frac{n}{2^3}\log\frac{n}{2^3}\right)\right] + n\log\frac{n}{2^2} + n\log\frac{n}{2} + n\log n$$

$$T\left(\frac{n}{2^3}\right) = 2^4 T\left(\frac{n}{2^4}\right) + n\log\frac{n}{2^3} + n\log\frac{n}{2^2} + n\log\frac{n}{2} + n\log n$$

$$\Rightarrow 2^k T\left(\frac{n}{2^k}\right) + n\log\frac{n}{2^{k-1}} + n\log\frac{n}{2^{k-2}} \cdots n\log\frac{n}{2^1} + n\log n$$

$$\Rightarrow \frac{n}{2^k} = 1$$

$$\boxed{n = 2^k} \quad \text{and} \quad \boxed{k = \log n}$$

$$\Rightarrow n\cdot(1) + n\left[\log\frac{n}{2^{k-1}} + \log\frac{n}{2^{k-2}} \cdots \log\frac{n}{2} + \log\frac{n}{2^0}\right]$$

$$\therefore \quad \frac{n}{2^{k-1}} \Rightarrow \frac{n}{\frac{2^k}{2}} \Rightarrow \frac{2n}{2^k}$$

$$\because 2^k = n$$

$$\Rightarrow \frac{2n}{n} \Rightarrow 2$$

$$\frac{n}{2^{k-1}} = 2$$

$$\Rightarrow n + n\left[\log 2 + \log 4 + \log 8 + \cdots\cdots \log n/2 + \log n\right]$$

$$\Rightarrow n + n\left[\log 2 + \log 2^2 + \log 2^3 + \cdots \log n/2 + \log n\right]$$

$$\Rightarrow n + n\left[1 + 2 + 3 + 4 + \cdots\cdots + \log n-1 + \log n\right]$$

$$\log n/2 = \log n - \log 2$$
$$= \log n - 1$$

$$\Rightarrow n + n\left[\frac{\log n(\log n + 1)}{2}\right]$$

$$\Rightarrow n + n\left[\frac{\log n \times \log n + \log n}{2}\right]$$

$$\Rightarrow n + \frac{n(\log n)^2 + n\log n}{2}$$

$$\Rightarrow n + n(\log n)^2 + n\log n$$

$$O(n(\log n)^2) \Rightarrow O(n\log^2 n)$$

$$T(n) = 8T\left(\frac{n}{2}\right) + n^2 \longrightarrow (1)$$

$$T(n) = 8\left[8T\left(\frac{n}{4}\right) + \left(\frac{n}{2}\right)^2\right] + n^2$$

$$T(n) = 8^2 T\left(\frac{n}{4}\right) + 2n^2 + n^2$$

$$T(n) = 8^2\left[8T\left(\frac{n}{8}\right) + \left(\frac{n}{4}\right)^2\right] + 2n^2 + n^2$$

$$= 8^3 T\left(\frac{n}{8}\right) + 4n^2 + 2n^2 + n^2$$

$$T(n) = 8^3\left[8T\left(\frac{n}{16}\right) + \left(\frac{n}{8}\right)^2\right] + 4n^2 + 2n^2 + n^2$$

$$= 8^4 T\left(\frac{n}{16}\right) + 8n^2 + 4n^2 + 2n^2 + n^2$$

$$\vdots$$

$$8^k T\left(\frac{n}{2^k}\right) + (k-1)n^2 + (k-2)n^2 + (k-1)n^2$$

$$\Rightarrow 8^k T\left(\frac{n}{2^k}\right) + 2^3 n^2 + 2^2 n^2 + 2^1 n^2 + 2^0 n^2$$

$$\Rightarrow 8^k T\left(\frac{n}{2^k}\right) + 2^{(k-1)} n^2 + 2^{(k-2)} n^2 + \cdots 2^1 n^2 + 2^0 n^2$$

$$\Rightarrow (2^3)^k = (8)^k \Rightarrow (2^k)^3$$

$n/2^k = 1$

$n = 2^k$

$k = \log_2 n$

$\Rightarrow (2^k)^3 + n^2 (2^0 + 2^1 + 2^2 - - - - + 2^{k-1})$ ——— (*)

G.P Series:

$Sum = a\frac{(r^n - 1)}{r - 1}$

$a = 2^0 = 1$ here $n = k$

$r = 2$

$Sum = 1\frac{(2^k - 1)}{2 - 1}$

(*) $\Rightarrow n^3 + n^2 \left((1)\frac{n-1}{1}\right)$

$n^3 + n^2 (n - 1)$

$n^3 + n^3 - 1$

$\Rightarrow 2n^3 - 1$

$O(n^3)$

Q10. True / False with Justification [2+2+2+2+2 Marks]

(a) f(n) + O(f(n)) = Θ(f(n)) → True

Let h(n) = O(f(n)), so $0 \le h(n) \le c \cdot f(n)$ for all $n \ge n_0$. Then:

$$f(n) \le f(n) + h(n) \le f(n) + c \cdot f(n) = (1 + c) \cdot f(n)$$

So f(n) + h(n) is bounded below by $1 \cdot f(n)$ and above by $(1 + c) \cdot f(n)$. Hence it is Θ(f(n)).

Example: $n^2 + 3n = \Theta(n^2)$, since $3n = O(n^2)$.

(b) If f(n) = O(g(n)) and g(n) = O(h(n)), then f(n) = O(h(n)) → True (Transitivity)

$f(n) \le c_1 \cdot g(n)$ for $n \ge n_1$, and $g(n) \le c_2 \cdot h(n)$ for $n \ge n_2$. For $n \ge \max(n_1, n_2)$:

$$f(n) \le c_1 \cdot g(n) \le c_1 c_2 \cdot h(n)$$

So f(n) = O(h(n)) with $c = c_1c_2$ and $n_0 = \max(n_1, n_2)$.

Example: $n = O(n^2)$ and $n^2 = O(n^3)$, so $n = O(n^3)$.

(c) If f(n) = Θ(g(n)), then f(n) + g(n) = Θ(g(n)) → True

Since $f(n) = \Theta(g(n))$, we have $c_1 \cdot g(n) \le f(n) \le c_2 \cdot g(n)$ for $n \ge n_0$. Adding g(n) to each part:

$$(1 + c_1) \cdot g(n) \le f(n) + g(n) \le (1 + c_2) \cdot g(n)$$

So $f(n) + g(n) = \Theta(g(n))$.

Example: $f(n) = 3n^2$, $g(n) = n^2$. Then $f(n) + g(n) = 4n^2 = \Theta(n^2)$.

(d) $n^2 = O(n^3)$ → True

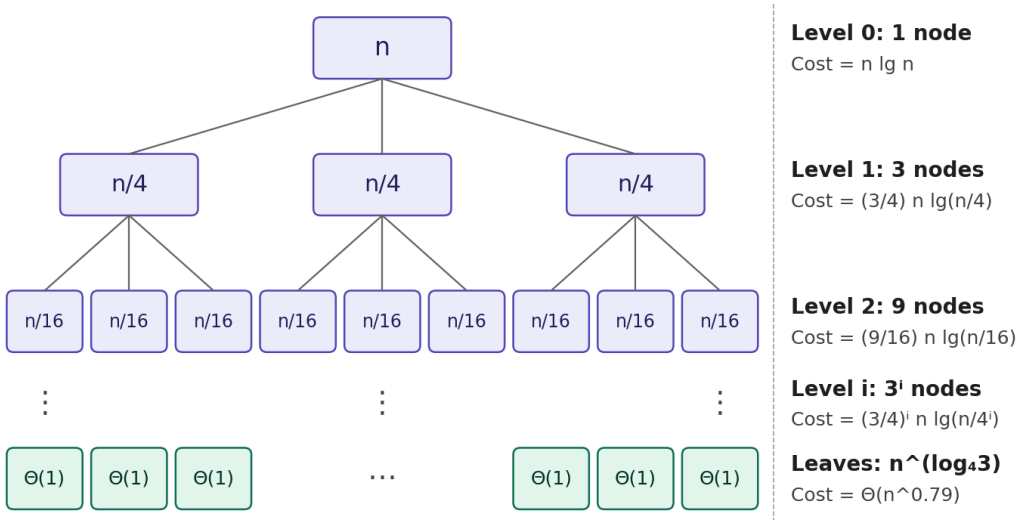
For $n \ge 1$, $n^2 \le n \cdot n^2 = n^3$. So $n^2 \le 1 \cdot n^3$ for all $n \ge 1$. **$c = 1$, $n_0 = 1$.**

(e) $n^3 = O(n^2)$ → False

Suppose $n^3 \le c \cdot n^2$ for all $n \ge n_0$. Dividing both sides by n^2 gives $n \le c$ for all $n \ge n_0$. This is impossible, because n grows without bound; for example, $n = \max(n_0, c) + 1$ violates the inequality. Therefore no constants c and n_0 exist, so $n^3 \ne O(n^2)$.

Q11. Recursion Tree Method [3.5+3.5 Marks]

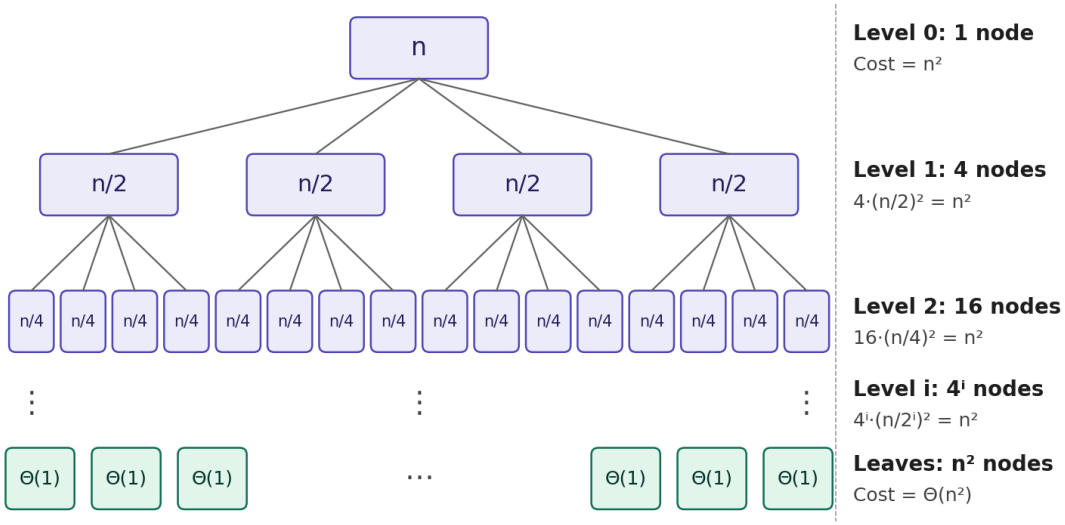
(a) T(n) = 3T(n/4) + n log n



Height $h = \log_4 n$ · costs shrink by 3/4 per level · root dominates

$T(n) \le 4\, n \lg n + \Theta(n^{0.79}) = \Theta(n \log n)$

(b) T(n) = 4T(n/2) + n²



Height $h = \log_2 n$ · every level costs n^2 · all levels contribute equally

$T(n) = n^2(\log_2 n + 1) = \Theta(n^2 \log n)$

Q12. Substitution Method [3.5+3.5 Marks]

(a) T(n) = 2T(n/2) + n log n

Guess: T(n) = Θ(n log² n) (The tree has log n levels, and each level costs about n log n.)

Claim: T(n) ≤ c · n log² n

Induction hypothesis: Assume the claim is true for n/2:

$$T(n/2) \le c\,(n/2)\log^2(n/2)$$

Substitute into the recurrence:

$$\begin{aligned} T(n) &\le 2 \cdot c(n/2)\log^2(n/2) + n\log n \\ &= c\,n\,(\log n - 1)^2 + n\log n \\ &= c\,n\log^2 n - 2c\,n\log n + c\,n + n\log n \\ &= c\,n\log^2 n - [(2c-1)\,n\log n - c\,n] \\ &\le c\,n\log^2 n \end{aligned}$$

The last step works when (2c − 1) log n ≥ c. For n ≥ 2 we have log n ≥ 1, so this holds whenever **c ≥ 1**.

Base case (n = 2): Let T(1) = d. T(2) = 2d + 2 ≤ c · 2 · log² 2 = 2c, which is true if c ≥ d + 1.

Constants: c = d + 1, $n_0 = 2$

Result: T(n) = O(n log² n), so **T(n) = Θ(n log² n)**

(b) T(n) = 2T(n/4) + √n

Guess: T(n) = Θ(√n log n) (Every level of the tree costs √n, and there are $\log_4 n$ levels.)

Claim: T(n) ≤ c √n log n

Induction hypothesis: Assume the claim is true for n/4:

$$T(n/4) \leq c\ \sqrt{(n/4)}\ \log(n/4)$$

Inductive step:

$$\begin{aligned} T(n) &\leq 2 \cdot c\ (\sqrt{n}/2)(\log n - 2) + \sqrt{n} \\ &= c\ \sqrt{n} \log n - 2c\ \sqrt{n} + \sqrt{n} \\ &= c\ \sqrt{n} \log n - (2c - 1)\ \sqrt{n} \\ &\leq c\ \sqrt{n} \log n, \text{ when } c \geq 1/2 \end{aligned}$$

Base case (n = 4): Let T(1) = d. T(4) = 2d + 2 ≤ c · √4 · log 4 = 4c, which is true if c ≥ (d + 1)/2.

Constants: c = (d + 1)/2, $n_0 = 4$

Result: T(n) = Θ(√n log n)

Why the log factor is needed: If we guess T(n) ≤ c √n without the log, substitution gives:

$$T(n) \leq 2c\ \sqrt{(n/4)} + \sqrt{n} = c\ \sqrt{n} + \sqrt{n}$$

This is always more than c√n, so the proof fails. The reason is that every level of the tree adds √n, and there are log n levels. The log factor is needed to count all of them.

Solution